

Java Collections: Batch 2

Twelve Demonstration Programs

Program 1: ArrayDequeExample

Question

Write a Java program to demonstrate ArrayDeque operations at both ends, including insertion, inspection, and removal.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `ArrayDequeExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create an empty ArrayDeque of integers.
2. Add 20 and 10 at the front, then 30 and 40 at the back.
3. Offer 5 at the front and 50 at the back.
4. Display the first and last elements using `peekFirst` and `peekLast`.
5. Remove the first and last elements with `pollFirst` and `pollLast`, displaying the deque after each removal.

UML Class Diagram

ArrayDequeExample

<code>main(args: String[]): void</code>
Uses: <code>ArrayDeque<Integer></code>

CODE: SRC

```
import java.util.ArrayDeque;

public class ArrayDequeExample {
    public static void main(String[] args) {

        ArrayDeque<Integer> numbers = new ArrayDeque<>();

        // addFirst()
        numbers.addFirst(20);
        numbers.addFirst(10);
        System.out.println("After addFirst: " + numbers);

        // addLast()
        numbers.addLast(30);
        numbers.addLast(40);
        System.out.println("After addLast: " + numbers);

        // offerFirst()
        numbers.offerFirst(5);
        System.out.println("After offerFirst: " + numbers);

        // offerLast()
        numbers.offerLast(50);
        System.out.println("After offerLast: " + numbers);

        // peekFirst()
        System.out.println("First element: " + numbers.peekFirst());

        // peekLast()
        System.out.println("Last element: " + numbers.peekLast());

        // pollFirst()
        System.out.println("Removed first: " + numbers.pollFirst());
        System.out.println("After pollFirst: " + numbers);

        // pollLast()
        System.out.println("Removed last: " + numbers.pollLast());
        System.out.println("After pollLast: " + numbers);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac ArrayDequeExample.java
java ArrayDequeExample
```

Output

```
After addFirst: [10, 20]
After addLast: [10, 20, 30, 40]
After offerFirst: [5, 10, 20, 30, 40]
After offerLast: [5, 10, 20, 30, 40, 50]
First element: 5
Last element: 50
Removed first: 5
After pollFirst: [10, 20, 30, 40, 50]
Removed last: 50
After pollLast: [10, 20, 30, 40]
```

Program 2: ArrayListExample

Question

Write a Java program to demonstrate common ArrayList operations, including insertion, access, replacement, removal, search, sorting, and clearing.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `ArrayListExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create an ArrayList and add four integers.
2. Insert 30 at index 2 and display the list.
3. Read the value at index 1, then replace the first value with 50.
4. Remove the value at index 3, then remove the Integer value 10.
5. Check membership, size, and empty status.
6. Add 40, find its first and last indexes, sort the list, and display it.
7. Clear the list and display the result.

UML Class Diagram

ArrayListExample

<code>main(args: String[]): void</code>

<code>Uses: ArrayList<Integer></code>

CODE: SRC

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<>();

        // add()
        numbers.add(25);
        numbers.add(10);
        numbers.add(40);
        numbers.add(15);

        System.out.println("Numbers: " + numbers);

        // add(index, element)
        numbers.add(2, 30);
        System.out.println("After inserting 30: " + numbers);

        // get()
        System.out.println("Element at index 1: " + numbers.get(1));

        // set()
        numbers.set(0, 50);
        System.out.println("After replacing first element: " + numbers);

        // remove(index)
        numbers.remove(3);
        System.out.println("After removing index 3: " + numbers);

        // remove(object)
        numbers.remove(Integer.valueOf(10));
        System.out.println("After removing 10: " + numbers);

        // contains()
        System.out.println("Contains 40? " + numbers.contains(40));

        // size()
        System.out.println("Size: " + numbers.size());

        // isEmpty()
        System.out.println("Is empty? " + numbers.isEmpty());

        // indexOf()
        numbers.add(40);
        System.out.println("First index of 40: " + numbers.indexOf(40));

        // lastIndexOf()
        System.out.println("Last index of 40: " + numbers.lastIndexOf(40));

        // sort()
        numbers.sort(null);
        System.out.println("Sorted list: " + numbers);

        // clear()
        numbers.clear();
        System.out.println("After clear: " + numbers);
    }
}
```

}

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac ArrayListExample.java
java ArrayListExample
```

Output

```
Numbers: [25, 10, 40, 15]
After inserting 30: [25, 10, 30, 40, 15]
Element at index 1: 10
After replacing first element: [50, 10, 30, 40, 15]
After removing index 3: [50, 10, 30, 15]
After removing 10: [50, 30, 15]
Contains 40? false
Size: 3
Is empty? false
First index of 40: 3
Last index of 40: 3
Sorted list: [15, 30, 40, 50]
After clear: []
```

Program 3: HashMapExample

Question

Write a Java program to demonstrate HashMap operations for an inventory, including insertion, lookup, removal, membership checks, views, and clearing.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `HashMapExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a HashMap from product names to quantities.
2. Insert four product and quantity pairs.
3. Display the map and retrieve the quantity for Mouse.
4. Remove Printer, then check whether Monitor and quantity 25 are present.
5. Display keys, values, entries, size, and empty status.
6. Use `getOrDefault` to query an absent product.
7. Clear the map and display it.

UML Class Diagram

HashMapExample

<code>main(args: String[]): void</code>
Uses: <code>HashMap<String, Integer></code>

CODE: SRC

```
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {

        HashMap<String, Integer> inventory = new HashMap<>();

        // put()
        inventory.put("Keyboard", 15);
        inventory.put("Mouse", 25);
        inventory.put("Monitor", 8);
        inventory.put("Printer", 4);

        System.out.println("Inventory: " + inventory);

        // get()
        System.out.println("Mouse quantity: " + inventory.get("Mouse"));

        // remove()
        inventory.remove("Printer");
        System.out.println("After removing Printer: " + inventory);

        // containsKey()
        System.out.println("Has Monitor? "
            + inventory.containsKey("Monitor"));

        // containsValue()
        System.out.println("Has quantity 25? "
            + inventory.containsValue(25));

        // keySet()
        System.out.println("Products: " + inventory.keySet());

        // values()
        System.out.println("Quantities: " + inventory.values());

        // entrySet()
        System.out.println("Entries: " + inventory.entrySet());

        // size()
        System.out.println("Number of products: " + inventory.size());

        // isEmpty()
        System.out.println("Is inventory empty? " + inventory.isEmpty());

        // getOrDefault()
        System.out.println("Speaker quantity: "
            + inventory.getOrDefault("Speaker", 0));

        // clear()
        inventory.clear();
        System.out.println("After clear: " + inventory);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac HashMapExample.java
java HashMapExample
```

Output

Note: HashMap does not guarantee iteration order. The order shown for products, keys, values, and entries is one possible order and may differ when the program runs.

```
Inventory: {Keyboard=15, Mouse=25, Monitor=8, Printer=4}
Mouse quantity: 25
After removing Printer: {Keyboard=15, Mouse=25, Monitor=8}
Has Monitor? true
Has quantity 25? true
Products: [Keyboard, Mouse, Monitor]
Quantities: [15, 25, 8]
Entries: [Keyboard=15, Mouse=25, Monitor=8]
Number of products: 3
Is inventory empty? false
Speaker quantity: 0
After clear: {}
```

Program 4: HashtableExample

Question

Write a Java program to demonstrate Hashtable operations, including insertion, lookup, membership checks, removal, enumeration, and size.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `HashtableExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a Hashtable mapping countries to capitals.
2. Insert four country and capital pairs.
3. Display the table and look up Japan.
4. Check for India and Paris, then remove Brazil.
5. Enumerate and print the country names, then enumerate and print the capitals.
6. Display the number of entries and whether the table is empty.

UML Class Diagram

HashtableExample	
main(args: String[]): void	
Uses: Hashtable<String, String>, Enumeration<String>	

CODE: SRC

```
import java.util.Hashtable;
import java.util.Enumeration;

public class HashtableExample {
    public static void main(String[] args) {

        Hashtable<String, String> capitals = new Hashtable<>();

        // put()
        capitals.put("India", "New Delhi");
        capitals.put("Japan", "Tokyo");
        capitals.put("France", "Paris");
        capitals.put("Brazil", "Brasilia");

        System.out.println("Capital Map: " + capitals);

        // get()
        System.out.println("Capital of Japan: "
            + capitals.get("Japan"));

        // containsKey()
        System.out.println("India present? "
            + capitals.containsKey("India"));

        // containsValue()
        System.out.println("Paris present? "
            + capitals.containsValue("Paris"));

        // remove()
        capitals.remove("Brazil");
        System.out.println("After removing Brazil: " + capitals);

        // keys()
        System.out.println("Countries:");
        Enumeration<String> countryNames = capitals.keys();

        while (countryNames.hasMoreElements()) {
            System.out.println(countryNames.nextElement());
        }

        // elements()
        System.out.println("Capitals:");
        Enumeration<String> capitalNames = capitals.elements();

        while (capitalNames.hasMoreElements()) {
            System.out.println(capitalNames.nextElement());
        }

        // size()
        System.out.println("Number of countries: " + capitals.size());

        // isEmpty()
        System.out.println("Is Hashtable empty? " + capitals.isEmpty());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac HashtableExample.java
java HashtableExample
```

Output

Note: Hashtable does not guarantee iteration order. The map order and enumeration order shown here are illustrative and may differ when the program runs.

```
Capital Map: {Japan=Tokyo, France=Paris, Brazil=Brasilia, India=New Delhi}
Capital of Japan: Tokyo
India present? true
Paris present? true
After removing Brazil: {Japan=Tokyo, France=Paris, India=New Delhi}
Countries:
Japan
France
India
Capitals:
Tokyo
Paris
New Delhi
Number of countries: 3
Is Hashtable empty? false
```

Program 5: LinkedHashMapExample

Question

Write a Java program to demonstrate LinkedHashMap operations and insertion-order iteration for player data.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `LinkedHashMapExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a LinkedHashMap from jersey numbers to player names.
2. Insert four player entries.
3. Display the map and retrieve the player at number 18.
4. Remove number 33 and display the updated map.
5. Check whether number 45 exists.
6. Display the jersey numbers, player names, and entries.

UML Class Diagram

LinkedHashMapExample

<code>main(args: String[]): void</code>
Uses: <code>LinkedHashMap<Integer, String></code>

CODE: SRC

```
import java.util.LinkedHashMap;

public class LinkedHashMapExample {
    public static void main(String[] args) {

        LinkedHashMap<Integer, String> players = new LinkedHashMap<>();

        // put()
        players.put(7, "Dhoni");
        players.put(18, "Virat");
        players.put(45, "Rohit");
        players.put(33, "Hardik");

        System.out.println("Players: " + players);

        // get()
        System.out.println("Player with number 18: "
            + players.get(18));

        // remove()
        players.remove(33);
        System.out.println("After removing number 33: " + players);

        // containsKey()
        System.out.println("Number 45 exists? "
            + players.containsKey(45));

        // keySet()
        System.out.println("Jersey numbers: " + players.keySet());

        // values()
        System.out.println("Player names: " + players.values());

        // entrySet()
        System.out.println("Player entries: " + players.entrySet());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac LinkedHashMapExample.java  
java LinkedHashMapExample
```

Output

```
Players: {7=Dhoni, 18=Virat, 45=Rohit, 33=Hardik}  
Player with number 18: Virat  
After removing number 33: {7=Dhoni, 18=Virat, 45=Rohit}  
Number 45 exists? true  
Jersey numbers: [7, 18, 45]  
Player names: [Dhoni, Virat, Rohit]  
Player entries: [7=Dhoni, 18=Virat, 45=Rohit]
```

Program 6: LinkedHashSetExample

Question

Write a Java program to demonstrate LinkedHashSet operations, showing uniqueness and insertion-order iteration.

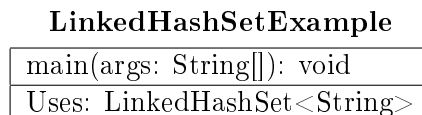
Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `LinkedHashSetExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a LinkedHashSet of colors.
2. Add Red, Blue, Green, Yellow, and Blue again.
3. Display the set to observe that duplicates are ignored and insertion order is kept.
4. Check whether Green and Black are present.
5. Display the number of unique colors.
6. Remove Red, display the set, then clear it and display the result.

UML Class Diagram



CODE: SRC

```
import java.util.LinkedHashSet;

public class LinkedHashSetExample {
    public static void main(String[] args) {

        LinkedHashSet<String> colors = new LinkedHashSet<>();

        // add()
        colors.add("Red");
        colors.add("Blue");
        colors.add("Green");
        colors.add("Yellow");
        colors.add("Blue");

        System.out.println("Colors: " + colors);

        // contains()
        System.out.println("Contains Green? " + colors.contains("Green"));
        System.out.println("Contains Black? " + colors.contains("Black"));

        // size()
        System.out.println("Number of colors: " + colors.size());

        // remove()
        colors.remove("Red");
        System.out.println("After removing Red: " + colors);

        // clear()
        colors.clear();
        System.out.println("After clear: " + colors);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac LinkedHashSetExample.java  
java LinkedHashSetExample
```

Output

```
Colors: [Red, Blue, Green, Yellow]  
Contains Green? true  
Contains Black? false  
Number of colors: 4  
After removing Red: [Blue, Green, Yellow]  
After clear: []
```

Program 7: LinkedListExample

Question

Write a Java program to demonstrate LinkedList operations, including indexed access, end operations, queue-style methods, and removals.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `LinkedListExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a LinkedList of station names and add Vizag, Vijayawada, and Guntur.
2. Add Srikakulam at the front and Nellore at the end.
3. Read the station at index 2, and display the first and last stations.
4. Remove the item at index 1, then remove Guntur by value.
5. Remove and display the first and last elements.
6. Offer Tirupati, inspect the head with peek, poll it, and display the final list.

UML Class Diagram



CODE: SRC

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {

        LinkedList<String> stations = new LinkedList<>();

        // add()
        stations.add("Vizag");
        stations.add("Vijayawada");
        stations.add("Guntur");

        System.out.println("Stations: " + stations);

        // addFirst()
        stations.addFirst("Srikakulam");
        System.out.println("After addFirst: " + stations);

        // addLast()
        stations.addLast("Nellore");
        System.out.println("After addLast: " + stations);

        // get()
        System.out.println("Station at index 2: " + stations.get(2));

        // getFirst()
        System.out.println("First station: " + stations.getFirst());

        // getLast()
        System.out.println("Last station: " + stations.getLast());

        // remove(index)
        stations.remove(1);
        System.out.println("After removing index 1: " + stations);

        // remove(object)
        stations.remove("Guntur");
        System.out.println("After removing Guntur: " + stations);

        // removeFirst()
        System.out.println("Removed first: " + stations.removeFirst());

        // removeLast()
        System.out.println("Removed last: " + stations.removeLast());

        // offer()
        stations.offer("Tirupati");
        System.out.println("After offer: " + stations);

        // peek()
        System.out.println("First element using peek: " + stations.peek());

        // poll()
        System.out.println("Removed using poll: " + stations.poll());

        System.out.println("Final list: " + stations);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac LinkedListExample.java
java LinkedListExample
```

Output

```
Stations: [Vizag, Vijayawada, Guntur]
After addFirst: [Srikakulam, Vizag, Vijayawada, Guntur]
After addLast: [Srikakulam, Vizag, Vijayawada, Guntur, Nellore]
Station at index 2: Vijayawada
First station: Srikakulam
Last station: Nellore
After removing index 1: [Srikakulam, Vijayawada, Guntur, Nellore]
After removing Guntur: [Srikakulam, Vijayawada, Nellore]
Removed first: Srikakulam
Removed last: Nellore
After offer: [Vijayawada, Tirupati]
First element using peek: Vijayawada
Removed using poll: Vijayawada
Final list: [Tirupati]
```

Program 8: PriorityQueueExample

Question

Write a Java program to demonstrate PriorityQueue insertion, inspection, removal, membership, and size operations.

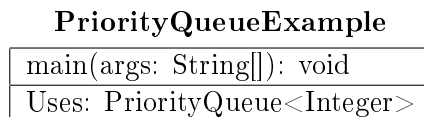
Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `PriorityQueueExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create an empty PriorityQueue of integers.
2. Add 5, 2, and 8, then display the queue.
3. Offer 1 and display the queue.
4. Read the head with peek.
5. Poll the head and display the queue.
6. Remove 8, check whether 5 is present, and display the queue size.

UML Class Diagram



CODE: SRC

```
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {

        PriorityQueue<Integer> patients = new PriorityQueue<>();

        // add()
        patients.add(5);
        patients.add(2);
        patients.add(8);

        System.out.println("Priority Queue: " + patients);

        // offer()
        patients.offer(1);
        System.out.println("After offer: " + patients);

        // peek()
        System.out.println("Highest priority: " + patients.peek());

        // poll()
        System.out.println("Removed: " + patients.poll());
        System.out.println("After poll: " + patients);

        // remove()
        patients.remove(8);
        System.out.println("After removing 8: " + patients);

        // contains()
        System.out.println("Contains 5? " + patients.contains(5));

        // size()
        System.out.println("Queue size: " + patients.size());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac PriorityQueueExample.java  
java PriorityQueueExample
```

Output

Note: PriorityQueue iteration and its printed representation are not guaranteed to be fully sorted; the smallest item is the head for the natural integer ordering.

```
Priority Queue: [2, 5, 8]  
After offer: [1, 2, 8, 5]  
Highest priority: 1  
Removed: 1  
After poll: [2, 5, 8]  
After removing 8: [2, 5]  
Contains 5? true  
Queue size: 2
```

Program 9: StackExample

Question

Write a Java program to demonstrate Stack operations, including push, peek, pop, empty, and search.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as **StackExample.java**.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create an empty stack of integers.
2. Push four values onto the stack and display it.
3. Inspect the top value with peek.
4. Pop the top value and display the stack.
5. Check whether the stack is empty.
6. Search for 20 and 10; search positions count from the top starting at 1.

UML Class Diagram

StackExample

main(args: String[]): void
Uses: Stack<Integer>

CODE: SRC

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {

        Stack<Integer> plates = new Stack<>();

        // push()
        plates.push(10);
        plates.push(20);
        plates.push(30);
        plates.push(40);

        System.out.println("Stack: " + plates);

        // peek()
        System.out.println("Top plate: " + plates.peek());

        // pop()
        System.out.println("Removed plate: " + plates.pop());
        System.out.println("Stack after pop: " + plates);

        // empty()
        System.out.println("Is stack empty? " + plates.empty());

        // search()
        System.out.println("Position of 20: " + plates.search(20));
        System.out.println("Position of 10: " + plates.search(10));
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac StackExample.java  
java StackExample
```

Output

```
Stack: [10, 20, 30, 40]  
Top plate: 40  
Removed plate: 40  
Stack after pop: [10, 20, 30]  
Is stack empty? false  
Position of 20: 2  
Position of 10: 3
```

Program 10: TreeMapExample

Question

Write a Java program to demonstrate TreeMap operations, including sorted keys, lookup, removal, key navigation, and entry inspection.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `TreeMapExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a TreeMap mapping ranks to names.
2. Insert five rank and name pairs.
3. Display the sorted map and look up rank 3.
4. Remove rank 6, then check for rank 2 and the value Ravi.
5. Display the first and last keys.
6. Query higher, lower, ceiling, and floor keys.
7. Display all entries.

UML Class Diagram

TreeMapExample

main(args: String[]): void
Uses: TreeMap<Integer, String>

CODE: SRC

```
import java.util.TreeMap;

public class TreeMapExample {
    public static void main(String[] args) {

        TreeMap<Integer, String> ranks = new TreeMap<>();

        // put()
        ranks.put(4, "David");
        ranks.put(1, "Aman");
        ranks.put(3, "Ravi");
        ranks.put(2, "Kiran");
        ranks.put(6, "Neha");

        System.out.println("Rank list: " + ranks);

        // get()
        System.out.println("Name at rank 3: " + ranks.get(3));

        // remove()
        ranks.remove(6);
        System.out.println("After removing rank 6: " + ranks);

        // containsKey()
        System.out.println("Contains rank 2? "
            + ranks.containsKey(2));

        // containsValue()
        System.out.println("Contains Ravi? "
            + ranks.containsValue("Ravi"));

        // firstKey() and lastKey()
        System.out.println("First key: " + ranks.firstKey());
        System.out.println("Last key: " + ranks.lastKey());

        // higherKey() and lowerKey()
        System.out.println("Key higher than 2: "
            + ranks.higherKey(2));

        System.out.println("Key lower than 3: "
            + ranks.lowerKey(3));

        // ceilingKey() and floorKey()
        System.out.println("Ceiling key of 2: "
            + ranks.ceilingKey(2));

        System.out.println("Floor key of 5: "
            + ranks.floorKey(5));

        // entrySet()
        System.out.println("Entries: " + ranks.entrySet());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac TreeMapExample.java
java TreeMapExample
```

Output

```
Rank list: {1=Aman, 2=Kiran, 3=Ravi, 4=David, 6=Neha}
Name at rank 3: Ravi
After removing rank 6: {1=Aman, 2=Kiran, 3=Ravi, 4=David}
Contains rank 2? true
Contains Ravi? true
First key: 1
Last key: 4
Key higher than 2: 3
Key lower than 3: 2
Ceiling key of 2: 2
Floor key of 5: 4
Entries: [1=Aman, 2=Kiran, 3=Ravi, 4=David]
```

Program 11: TreeSetExample

Question

Write a Java program to demonstrate TreeSet operations, including sorted uniqueness, navigation, and endpoint removal.

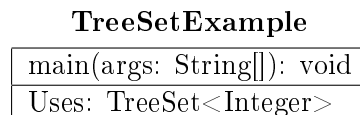
Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `TreeSetExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a TreeSet of integer scores.
2. Add five scores, including 78 twice.
3. Display the sorted set and remove 32.
4. Check whether 65 is present; display the first and last scores.
5. Query higher and lower values around 65, and ceiling and floor values around 70.
6. Remove and display the lowest and highest values using pollFirst and pollLast.

UML Class Diagram



CODE: SRC

```
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {

        TreeSet<Integer> scores = new TreeSet<>();

        // add()
        scores.add(45);
        scores.add(78);
        scores.add(32);
        scores.add(91);
        scores.add(65);
        scores.add(78);

        System.out.println("Scores: " + scores);

        // remove()
        scores.remove(32);
        System.out.println("After removing 32: " + scores);

        // contains()
        System.out.println("Contains 65? " + scores.contains(65));

        // first() and last()
        System.out.println("Lowest score: " + scores.first());
        System.out.println("Highest score: " + scores.last());

        // higher() and lower()
        System.out.println("Score higher than 65: " + scores.higher(65));
        System.out.println("Score lower than 65: " + scores.lower(65));

        // ceiling() and floor()
        System.out.println("Ceiling of 70: " + scores.ceiling(70));
        System.out.println("Floor of 70: " + scores.floor(70));

        // pollFirst()
        System.out.println("Removed lowest: " + scores.pollFirst());
        System.out.println("After pollFirst: " + scores);

        // pollLast()
        System.out.println("Removed highest: " + scores.pollLast());
        System.out.println("After pollLast: " + scores);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac TreeSetExample.java
java TreeSetExample
```

Output

```
Scores: [32, 45, 65, 78, 91]
After removing 32: [45, 65, 78, 91]
Contains 65? true
Lowest score: 45
Highest score: 91
Score higher than 65: 78
Score lower than 65: 45
Ceiling of 70: 78
Floor of 70: 65
Removed lowest: 45
After pollFirst: [65, 78, 91]
Removed highest: 91
After pollLast: [65, 78]
```

Program 12: VectorExample

Question

Write a Java program to demonstrate Vector operations, including adding elements, access, replacement, removal, size, capacity, and membership.

Steps for Execution

1. Open a Java editor or terminal.
2. Save the source code as `VectorExample.java`.
3. Compile and run it with the commands on the next page.
4. Observe the collection operations and their results.

Algorithm

1. Create a Vector of Double values and add three temperatures.
2. Append 36.4 with `addElement` and display the vector.
3. Read the value at index 1 and replace index 0 with 30.5.
4. Remove the value at index 2, then remove the element 36.4 by value.
5. Display the vector size, capacity, and whether it contains 30.5.

UML Class Diagram

VectorExample

<code>main(args: String[]): void</code>
Uses: <code>Vector<Double></code>

CODE: SRC

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {

        Vector<Double> temperatures = new Vector<>();

        // add()
        temperatures.add(32.5);
        temperatures.add(35.2);
        temperatures.add(31.8);

        System.out.println("Temperatures: " + temperatures);

        // addElement()
        temperatures.addElement(36.4);
        System.out.println("After addElement: " + temperatures);

        // get()
        System.out.println("Temperature at index 1: "
                           + temperatures.get(1));

        // set()
        temperatures.set(0, 30.5);
        System.out.println("After set: " + temperatures);

        // remove(index)
        temperatures.remove(2);
        System.out.println("After removing index 2: " + temperatures);

        // removeElement()
        temperatures.removeElement(36.4);
        System.out.println("After removeElement: " + temperatures);

        // size()
        System.out.println("Size: " + temperatures.size());

        // capacity()
        System.out.println("Capacity: " + temperatures.capacity());

        // contains()
        System.out.println("Contains 30.5? "
                           + temperatures.contains(30.5));
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac VectorExample.java  
java VectorExample
```

Output

```
Temperatures: [32.5, 35.2, 31.8]  
After addElement: [32.5, 35.2, 31.8, 36.4]  
Temperature at index 1: 35.2  
After set: [30.5, 35.2, 31.8, 36.4]  
After removing index 2: [30.5, 35.2, 36.4]  
After removeElement: [30.5, 35.2]  
Size: 2  
Capacity: 10  
Contains 30.5? true
```